

Bioinformatics session

4th annual workshop on
bioinformatics and variant
interpretation in InPreD

https://inpred.github.io/26-09_bioinfo_ws/



2. Containerization



A brief history

1979	<code>chroot</code> system call (changing the root directory of a process and its children to a new location in the filesystem) in Uni V7 considered as beginning of process isolation
2000	FreeBSD Jails allows administrators to partition FreeBSD computer system into several independent, smaller systems (<i>jails</i>) – with the ability to assign IP address for each system and configuration
2001	Linux VServer is jail mechanism that can partition resources, similar to FreeBSD Jails

2004	First public beta of Solaris Containers released - combines system resource controls and boundary separation provided by zones
2005	Open Virtuozzo is operating system-level virtualization technology for Linux which uses patched Linux kernel for virtualization, isolation, resource management and checkpointing
2006	Process Containers, launched by Google, was designed for limiting, accounting and isolating resource usage of collection of processes - renamed to <i>Control Groups (cgroups)</i> and merged into Linux kernel
2008	Linux Containers (LXC) was the first, most complete implementation of a Linux container manager - implemented using cgroups and Linux namespaces

2011	Warden from Cloud Foundry can isolate environments on any operating system, runs as a daemon and provides API for container management
2013	Let Me Contain That For You (LMCTFY) kicked off as open-source version of Google's container stack - providing Linux application containers
2013	Docker emerged
2016	Singularity (later Apptainer) was released

Docker

What is it? 🔍

- a technology for packaging an application with all of its dependencies into an independent executable unit (a container)
- the container image can be shipped and run consistently across different computing environments

Which benefits does it provide?



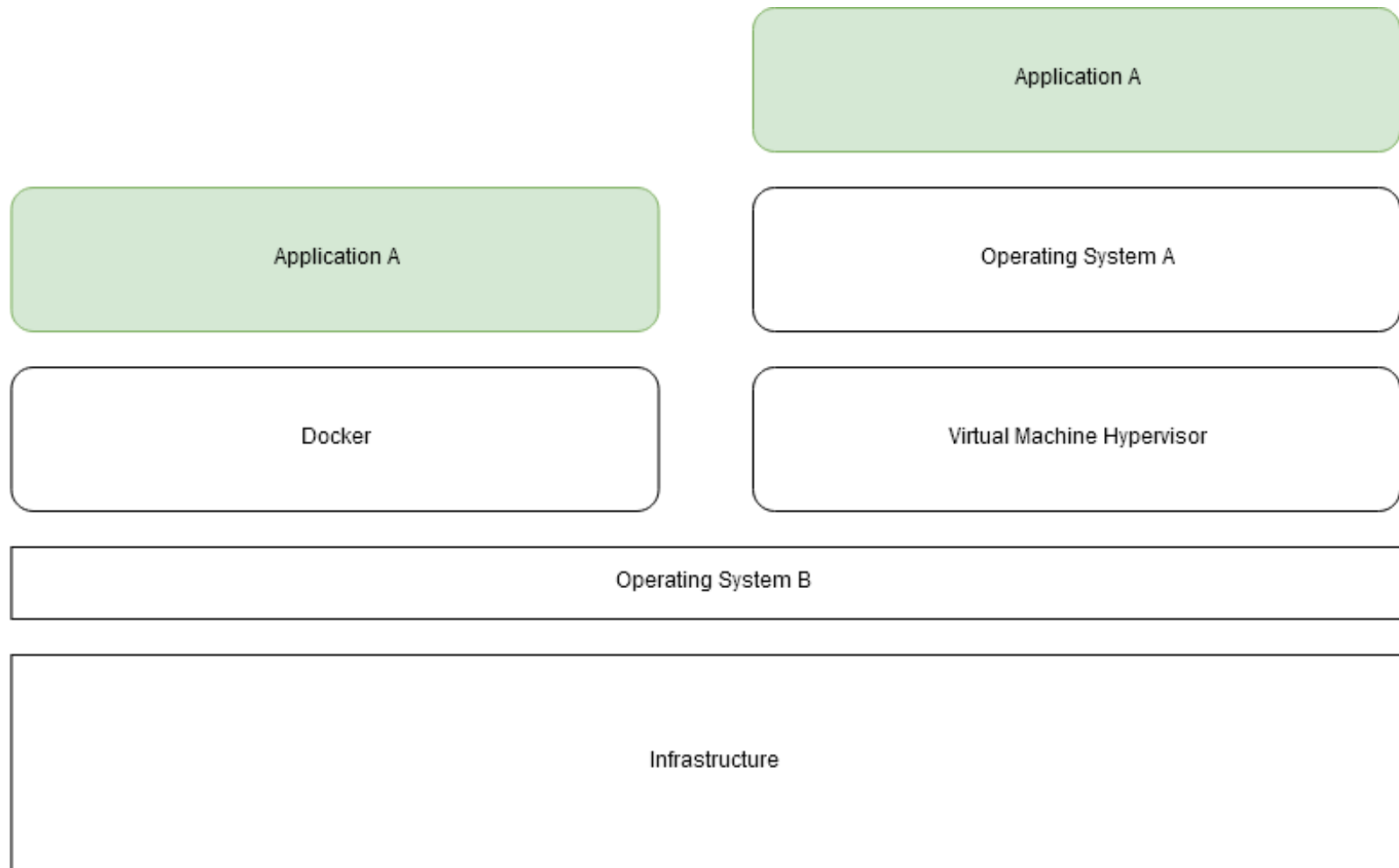
1. Consistency
2. Isolated environments
3. Portability
4. Efficiency

<https://scalablehuman.com/2023/10/24/the-fallacy-of-it-works-on-my-machine>



How is it different from virtual machines?

- containers use and share host OS's kernel, which makes them more lightweight and efficient
- virtual machines emulate entire physical machines, including the complete OS, making it possible to run multiple OS instances on a single physical machine



<https://dev.to/swikritit/docker-for-dummies-introduction-to-docker-5h67>

Key Concepts 🔑

1. **Image:** a standardized package that includes all files, binaries, libraries, and configurations necessary to run a given container
2. **Container:** a running instance of an image, operating in an isolated runtime environment
3. **Dockerfile:** instructions on how to build an image
4. **Docker Hub:** a public registry where developers can share their pre-built images

Key Components 🔑

1. **Docker Engine:** core, open-source technology that builds and runs containers
2. **Docker Client:** command-line tool that sends instructions to the Docker Daemon using REST APIs
3. **Docker Daemon (`dockerd`):** receives and processes API requests and calls container runtime
4. **Container Runtime (`containerd`):** turns static container image into running, isolated application on host OS; industry standard

Let's explore! 🌍

Start by going to https://github.com/InPreD/26-09_bioinfo_ws_docker_and_ci and create a fork.

The screenshot shows the GitHub interface for the repository '26-09_bioinfo_ws_docker_and_ci'. At the top, there are buttons for 'Edit Pins', 'Watch' (0), 'Fork' (0), and 'Star' (0). Below these, a dropdown menu is open, displaying the message 'You don't have any forks of this repository.' and a button labeled '+ Create a new fork', which is highlighted with a red rectangle. The repository is owned by 'marrip' and has a 'main' branch. A table of files is shown, including '.devcontainer', 'src/greeter', '.gitignore', 'LICENSE', 'README.md', and 'pyproject.toml'. The README section is expanded, showing the title '26-09_bioinfo_ws_docker_and_ci' and the description 'codespace template for docker and ci workshop'. On the right sidebar, there are sections for 'workshop' (with links to Readme, AGPL-3.0 license, Activity, Custom properties, 0 stars, 0 watching, 0 forks, Audit log, and Report repository), 'Releases' (with a link to 'Create a new release'), and 'Packages' (with a link to 'Publish your first package').

26-09_bioinfo_ws_docker_and_ci (Public)

Edit Pins Watch 0 Fork 0 Star 0

You don't have any forks of this repository.

+ Create a new fork

main 1 Branch 0 Tags

Go to file T Add file

marrip feat: add simple python project e4198b4 · 8 minutes ago 3 Commits

.devcontainer	chore: add devcontainer setup	9 minutes ago
src/greeter	feat: add simple python project	8 minutes ago
.gitignore	chore: add devcontainer setup	9 minutes ago
LICENSE	Initial commit	1 hour ago
README.md	Initial commit	1 hour ago
pyproject.toml	feat: add simple python project	8 minutes ago

README AGPL-3.0 license

26-09_bioinfo_ws_docker_and_ci

codespace template for docker and ci workshop

workshop

- Readme
- AGPL-3.0 license
- Activity
- Custom properties
- 0 stars
- 0 watching
- 0 forks
- Audit log
- Report repository

Releases

No releases published

[Create a new release](#)

Packages

No packages published

[Publish your first package](#)

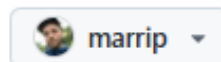
Create your own fork by clicking on **Create fork**.

Create a new fork

A *fork* is a copy of a repository. Forking a repository allows you to freely experiment with changes without affecting the original project.

Required fields are marked with an asterisk ().*

Owner *



Repository name *

/ 26-09_bioinfo_ws_docker_

✓ 26-09_bioinfo_ws_docker_and_ci is available.

By default, forks are named the same as their upstream repository. You can customize the name to distinguish it further.

Description

codespace template for docker and ci workshop

45 / 350 characters

☒ Copy the `main` branch only

Contribute back to InPreD/26-09_bioinfo_ws_docker_and_ci by adding your own branch. [Learn more.](#)

You are creating a fork in your personal account.

Create fork

In the forked repository, navigate to **Code** > **Codespaces** > **Create codespace on main** .

The screenshot shows a GitHub repository page for '26-09_bioinfo_ws_docker_and_ci' by user 'marrrip'. The repository is public and has 1 branch and 0 tags. A modal window is open over the repository, showing the 'Codespaces' tab. The modal indicates that no codespaces are currently set up for this repository and provides a button to 'Create codespace on main'. The repository's README is visible in the background, stating it is a 'codespace template for docker and ci workshop' under an AGPL-3.0 license. The right sidebar shows repository statistics: 0 stars, 0 watching, 0 forks, and no releases or packages published.

26-09_bioinfo_ws_docker_and_ci Public

Edit Pins Watch 0 Fork 0 Star 0

main 1 Branch 0 Tags

Go to file T Add file <> Code

marrrip feat: add simple python project

- .devcontainer chore: add devconta
- src/greeter feat: add simple pyth
- .gitignore chore: add devconta
- LICENSE Initial commit
- README.md Initial commit
- pyproject.toml feat: add simple pyth

Local Codespaces

Codespaces
Your workspaces in the cloud

No codespaces
You don't have any codespaces with this repository checked out

Create codespace on main

[Learn more about codespaces...](#)

Codespace usage for this repository is paid for by marrrip.

26-09_bioinfo_ws_docker_and_ci

codespace template for docker and ci workshop

AGPL-3.0 license

Readme

AGPL-3.0 license

Activity

Custom properties

0 stars

0 watching

0 forks

Audit log

Report repository

Releases

No releases published

[Create a new release](#)

Packages

No packages published

[Publish your first package](#)

Inside the terminal, run the following commands:

```
# check for running docker containers
$ docker ps
# check for existing images
$ docker images
# pull image
$ docker pull ubuntu:26.04
# run image
$ docker run ubuntu:26.04
```

Run an interactive container:

```
# start interactive container
$ docker run -it ubuntu:26.04
# print container os version
@ cat /etc/lsb-release
# exit container
@ exit
# print os version
$ lsb_release -a
```

Instruct docker to remove containers after use:

```
# check for any docker containers
$ docker ps -a
# remove all stopped containers
$ docker rm <CONTAINER ID>
# run docker with --rm flag
$ docker run -it --rm ubuntu:26.04
# exit container
@ exit
# check for all docker containers
$ docker ps -a
```

Remove the docker image:

```
# remove docker image
$ docker rmi ubuntu:26.04
# alternatively check for container image id
$ docker images
# and remove the image by using the id
$ docker rmi <ID>
```

Building an image

We start off by creating a `Dockerfile` in the root directory of our repository (`/workspaces/26-09_bioinfo_ws_docker_and_ci`). We add the following to our file:

```
FROM python:3.14-slim-trixie
RUN echo "Hello world!" > greetings.txt
```

And then we build and run our docker image:

```
# build tagged docker image
$ docker build . -t greeter:test
# run tagged docker image
$ docker run --rm greeter:test cat greetings.txt
```

Let us add a label to our `Dockerfile` to indicate who the author and maintainer is:

```
FROM python:3.14-slim-trixie
LABEL org.opencontainers.image.authors="martin.rippin@helse-bergen.no"
RUN echo "Hello world!" > greetings.txt
```

And then we build and inspect our docker image:

```
# build tagged docker image
$ docker build . -t greeter:test
# run tagged docker image
$ docker inspect greeter:test
```


Next, we are setting `cat greetings.txt` as a default command that is run whenever the container is started:

```
FROM python:3.14-slim-trixie
LABEL org.opencontainers.image.authors="martin.rippin@helse-bergen.no"
RUN echo "Hello world!" > greetings.txt
CMD ["cat", "greetings.txt"]
```

And then we build and run our docker image:

```
# build tagged docker image
$ docker build . -t greeter:test
# run tagged docker image
$ docker run --rm greeter:test
```

There is a small python application in this repository that we would like to include in our docker image:

```
FROM python:3.14-slim-trixie
LABEL org.opencontainers.image.authors="martin.rippin@helse-bergen.no"
WORKDIR /usr/src/greeter
COPY pyproject.toml ./
COPY src/ ./src/
RUN pip install --no-cache-dir .
CMD ["greeter"]
```

And then we build and run our docker image:

```
# build tagged docker image
$ docker build . -t greeter:test
# run tagged docker image
$ docker run --rm greeter:test
```

A full list of `Dockerfile` instructions can be found here:

<https://docs.docker.com/reference/dockerfile#overview>

Apptainer

What is it?

- container platform designed for ease-of-use on shared systems and in high performance computing (HPC) environments

How is it different from Docker?

Apptainer	Docker
without root-privileges by default	requires root privileges for most operations
SIF (Singularity Image Format) – immutable, portable, cryptographically signed	Docker/OCI images – layered file systems, mutable by default
can run Docker/OCI images	cannot run SIF images

Let's explore! 🌐

Similar to Docker, Apptainer provides a cli:

```
# check for any apptainer container
$ apptainer instance list -a
# pull image
$ apptainer pull docker://ubuntu:26.04
# check identity
$ whoami
# check identity in container
$ apptainer exec docker://ubuntu:26.04 whoami
```

The container image is saved as a `.sif` -file to the working directory.

Building an image

We are creating a file called `greeter.def` in the root of our repository and add the following lines:

```
Bootstrap: docker
From: python:3.14-slim-trixie

%post
    echo "Hello world!" > /usr/src/greetings.txt
```

And then we build and run our apptainer image:

```
# build apptainer image
$ apptainer build greeter.sif greeter.def
# execute apptainer image
$ apptainer exec greeter.sif cat /usr/src/greetings.txt
```

A full list of apptainer definition file sections can be found here:

https://apptainer.org/docs/user/main/definition_files.html#sections

3. Continuous Integration (CI)



What is it?

- automating integration of code changes from multiple contributors into single software project
- developers frequently merge code changes into central repository where automated tools are used to assert new code's correctness before integration (test, lint, build)

Which benefits does it provide?

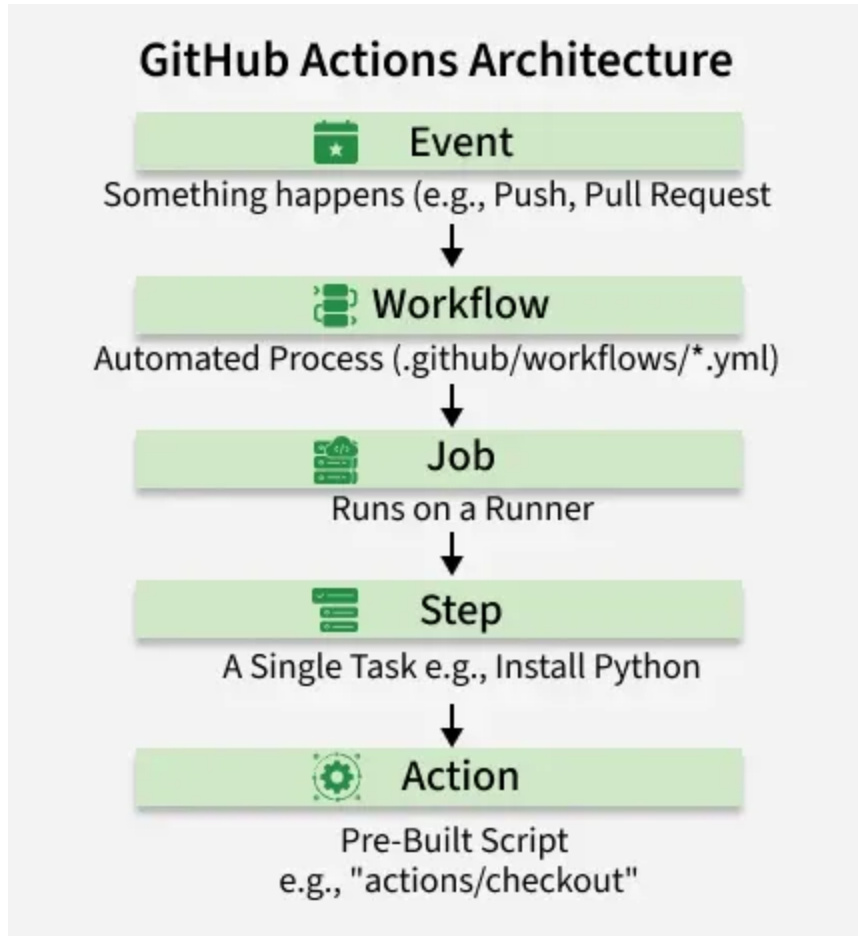
- scale up delivery output
- enables parallel work on features

GitHub actions

What is it? 🔍

- GitHub's continuous integration automation platform
- repetitive tasks are triggered based on code pushes, pull requests, or custom schedules

How does it work? 🤔



<https://www.geeksforgeeks.org/git/introduction-to-github-actions>

Workflow:

- `.yaml` -file located at `.github/workflows/` containing CI instructions

Event

- defined in workflow file
- trigger to start workflow, e.g. push, pull_request, schedule, etc.

Jobs

- defined in workflow file
- contains several tasks/*steps* and is running on a specific runner (server executing the code of the workflow)
- jobs run in parallel or can depend on each other, e.g. testing before building a container image

Step

- single task inside a job, e.g. installing python

GitHub Action

- reusable steps written by other developers that are freely available
- shorter workflows and avoiding unnecessary repetition

Example

```
name: Hello World
# event definition
on: [push]

jobs:
  # job definition
  say-hello:
    # runner definition
    runs-on: ubuntu-latest

    steps:
      # github action definition, more information about the checkout action at https://github.com/actions/checkout
      - name: Checkout Code
        uses: actions/checkout@v7

      # step definition
      - name: Run a Hello World Script
        run: |
          echo "Hello, world!"
          echo "The current repository branch is ${github.ref}"
```


Let's explore! 🌐

We are adding a workflow file to our repository:

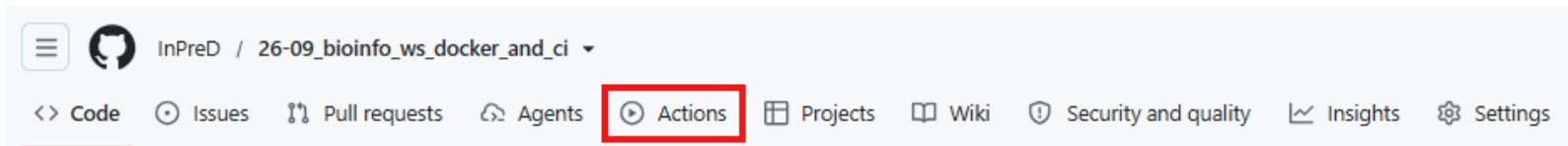
```
# create .github/workflows  
$ mkdir -p .github/workflows
```

Now we add `.github/workflows/hello_world.yaml` using the example file from the previous slide.

Now we commit our workflow doing:

```
# stage the workflow yaml file
$ git add .github/workflows/hello_world.yaml
# commit with commit message using appropriate git commit tag
$ git commit -m "ci: add hello world workflow"
# push the changes to the remote
$ git push
```

In the repository on GitHub, we navigate to **Actions**.



We see that the workflow was triggered by our last commit. To inspect we simply click on the workflow run named by our last commit message.

Actions

[New workflow](#)

All workflows

Hello World

Management

Caches

Attestations

Runners

Usage metrics

Performance metrics

Upcoming change to GitHub App installation token format

GitHub App installation tokens will soon use a new stateless format (ghs_...) and may be longer (~520 characters). Apps with hardcoded length assumptions may break.

Validate your apps and workflows with the per-request override header detailed in this [GitHub Changelog](#).

All workflows

Showing runs from all workflows

1 workflow run

Workflow ▾ Event ▾ Status ▾ Branch ▾ Actor ▾

ci: add hello world workflow

Hello World #3: Commit [2431e9b](#) pushed by [marrjp](#)

ci-branch

now
Queued

...

Our job `say-hello` was triggered. Select the job to inspect it.

← Hello World

✓ ci: add hello world workflow #3 Re-run all jobs ⋮

Summary

All jobs ⌵

✓ say-hello

Run details

🕒 Usage

📄 Workflow file

Triggered via push 1 minute ago	Status	Total duration	Artifacts
 marrjp pushed ↗ 2431e8b c1-branch	Success	<u>7s</u>	—

hello_world.yaml
on: push

✓ say-hello 3s

🔄 − +

We see several steps being run which we can expand by clicking on them. But most importantly, our workflow was successfully run! 🎉

say-hello

succeeded 2 minutes ago in 3s

Search logs

🔄 ⚙️

Set up job

15

1

Current runner version: '2.337.0'

2

▶ Runner Image Provisioner

9

▶ Operating System

13

▶ Runner Image

18

▶ GITHUB_TOKEN Permissions

39

Secret source: Actions

40

Prepare workflow directory

41

Prepare all required actions

42

Getting action download info

43

Download action repository 'actions/checkout@v7' (SHA:3d3c42e5aac5ba805825da76410c181273ba90b1)

44

Complete job name: say-hello

Checkout Code

05

1

▶ Run actions/checkout@v7

17

Syncing repository: InPreB/26-09_biointo_ws_docker_and_ci

18

▶ Getting Git version info

22

Temporarily overriding HOME='/home/runner/work/_temp/bb1914fe-6a83-4b02-9151-8ec6d2a108f6' before making global git config changes

23

Adding repository directory to the temporary git global config as a safe directory

24

/usr/bin/git config --global --add safe.directory /home/runner/work/26-09_biointo_ws_docker_and_ci/26-09_biointo_ws_docker_and_ci

25

Deleting the contents of '/home/runner/work/26-09_biointo_ws_docker_and_ci/26-09_biointo_ws_docker_and_ci'

26

▶ Determining repository object format

27

▶ Initializing the repository

44

▶ Disabling automatic garbage collection

46

▶ Setting up auth

61

▶ Fetching the repository

69

▶ Determining the checkout info

70

/usr/bin/git sparse-checkout disable

71

/usr/bin/git config --local --unset-all extensions.worktreeconfig

72

▶ Checking out the ref

76

/usr/bin/git log -1 --format=%H

77

2431e9b1e654231e21fe2f76d830e80a7c2a73a3

Run a Hello World Script

15

1

▶ Run echo "Hello, world!"

5

Hello, world!

6

The current repository branch is refs/heads/ci-branch

Post Checkout Code

05

Complete job

05

Let's use GitHub actions to lint our Dockerfile and build the image. We start by creating `.github/workflows/docker.yaml` and add the following:

```
name: Docker Lint and Build
on: [push]

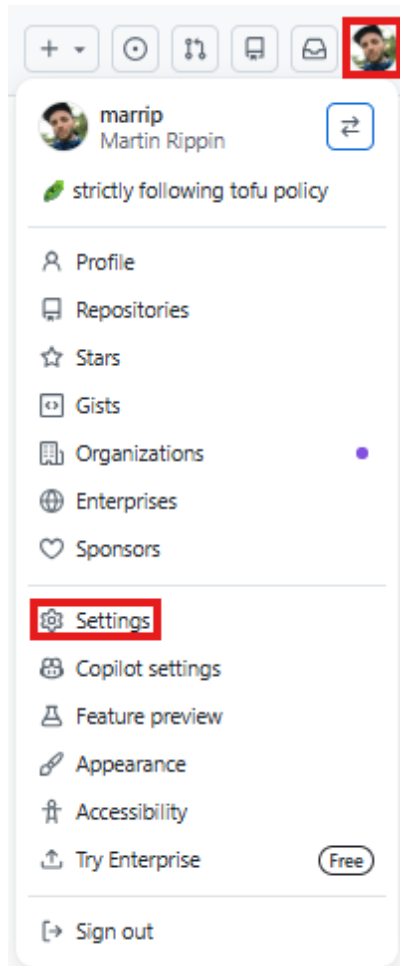
jobs:
  lint:
    name: Build Image
    runs-on: ubuntu-latest
    steps:
      # action to checkout repository
      - name: Check out the repo
        uses: actions/checkout@v7
      # action to use hadolint for linting the Dockerfile
      - name: Lint Dockerfile
        uses: hadolint/hadolint-action@v3.5.0
```

After adding the new workflow, we can commit both the `Dockerfile` and `.github/workflows/docker.yaml`:

```
# stage the Dockerfile and workflow yaml file
$ git add Dockerfile .github/workflows/docker.yaml
# commit with commit message using appropriate git commit tag
$ git commit -m "ci: add Dockerfile and docker workflow"
# push the changes to the remote
$ git push
```

We navigate to `Actions` on GitHub to check if hadolint runs successfully.

Building our docker image, we would also like to place it into the GitHub container registry (ghcr). To give the github action runner access to our personal registry, we need to create a personal access token (pat). Click on your avatar in the right corner and select **Settings** from the dropdown.



 Public profile

 Account

 Appearance

 Accessibility

 Notifications

Access

 Billing and licensing 

 Emails

 Password and authentication

 Sessions

 SSH and GPG keys

 Credentials

 Organizations

 Enterprises

 Teams

 Moderation 

Code, planning, and automation

 Repositories

 Codespaces

 Packages

 Copilot 

 Pages

 Saved replies

Security

 Code security

Integrations

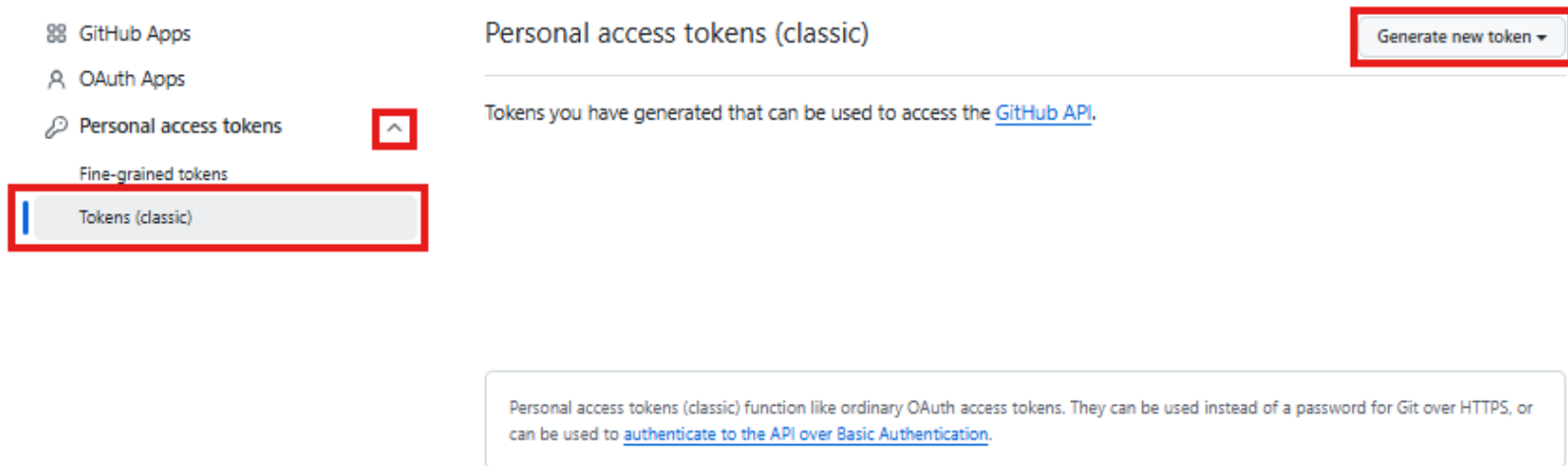
 Applications

 Scheduled reminders

Archives

 Security log

Expand Personal access tokens and select Tokens (classic) > Generate new token > Generate new token (classic).



Give the token a descriptive name `ghcr_push_token`, select an `Expiration` (7 days should be enough) and select the `write:packages` scope (will automatically select other necessary scopes)

New personal access token (classic)

Personal access tokens (classic) function like ordinary OAuth access tokens. They can be used instead of a password for Git over HTTPS, or can be used to [authenticate to the API over Basic Authentication](#).

Note

ghcr_push_token

What's this token for?

Expiration

📅 7 days (Sep 16, 2026) ▾

The token will expire on the selected date

Select scopes

Scopes define the access for personal tokens. [Read more about OAuth scopes](#).

☒ repo	Full control of private repositories
☒ repo:status	Access commit status
☒ repo_deployment	Access deployment status
☒ public_repo	Access public repositories
☒ repo:invite	Access repository invitations
☒ security_events	Read and write security events
☐ workflow	Update GitHub Action workflows
☒ write:packages	Upload packages to GitHub Package Registry
☒ read:packages	Download packages from GitHub Package Registry
☐ delete:packages	Delete packages from GitHub Package Registry

Scroll to the bottom of the page and confirm with **Generate token**.

- ☐ **admin:ssh_signing_key** Full control of public user SSH signing keys
- ☐ **write:ssh_signing_key** Write public user SSH signing keys
- ☐ **read:ssh_signing_key** Read public user SSH signing keys

Generate token

[Cancel](#)

Copy the token.

Personal access tokens (classic)

Generate new token ▾

Tokens you have generated that can be used to access the [GitHub API](#).

 Make sure to copy your personal access token now. You won't be able to see it again!

✓ ghp_



Delete

[!WARNING]

The token will only be accessible after creation, do not leave this site until you have completed adding it to your repository.

Now we expand our docker workflow by adding the build job below the lint job:

```
name: Docker Lint and Build
on: [push]

jobs:
  lint:
    ...
  build:
    name: Build Image
    runs-on: ubuntu-latest
    needs: lint
    steps:
      - name: Check out the repo
        uses: actions/checkout@v7
      # action to login to GitHub container registry
      - name: Login to GitHub container registry
        uses: docker/login-action@v4
        with:
          registry: ghcr.io # address to GitHub container registry
          username: ${github.actor} # the user triggering the workflow
          password: ${secrets.GHCR_PUSH_TOKEN} # The personal access token we have created earlier
      # action to build and push the image to GitHub container registry
      - name: Build and push image to GitHub container registry
        uses: docker/build-push-action@v5
        with:
          push: true
          tags: |
            ghcr.io/${github.actor}/greeter:latest
```